RecGenie

March 18, 2025

1 Content Based Filtering

Recommendation systems are a collection of algorithms used to recommend items to users based on information taken from the user. These systems have become ubiquitous, and can be commonly seen in online stores, movies databases and job finders. In this notebook, we will explore Content-based recommendation systems and implement a simple version of one using Python and the Pandas library.

1.0.1 Table of contents

2 Acquiring the Data

[2]:
```
import pandas as pd
import os
```

[3]:
```
#Storing the movie information into a pandas dataframe
# movies_df = pd.read_csv('movies.csv')
#Storing the user information into a pandas dataframe
# ratings_df = pd.read_csv('ratings.csv')

# movies_df.head()
```

[17]:
```
dataset_path = os.path.join(os.path.expanduser('~'), 'datasets', 'movies', '7')

credits_df = pd.read_csv(dataset_path + '/credits.csv')
# movies_meta_df = pd.read_csv(dataset_path + '/movies_metadata.csv')
# ratings_df = pd.read_csv(dataset_path + '/ratings.csv')
```

[217]:
```
dataset_path_2 = os.path.join(os.path.expanduser('~'), 'datasets', 'ml-32m')
ratings_df = pd.read_csv(dataset_path_2 + '/ratings.csv')
```

```
# links = pd.read_csv(dataset_path_2 + '/links.csv')
#
# tags = pd.read_csv(dataset_path_2 + '/tags.csv')
# tags.head(40)
```

[218]:
```
ratings_df.head()
```

[218]:

	userId	movieId	rating	timestamp
0	1	17	4.0	944249077
1	1	25	1.0	944250228
2	1	29	2.0	943230976
3	1	30	5.0	944249077
4	1	32	5.0	943228858

[15]:
```
ratings_2 = pd.read_csv(dataset_path_2 + '/ratings.csv')
ratings_2.head(10)
```

[15]:

	userId	movieId	rating	timestamp
0	1	17	4.0	944249077
1	1	25	1.0	944250228
2	1	29	2.0	943230976
3	1	30	5.0	944249077
4	1	32	5.0	943228858
5	1	34	2.0	943228491
6	1	36	1.0	944249008
7	1	80	5.0	944248943
8	1	110	3.0	943231119
9	1	111	5.0	944249008

[16]:
```
ratings_2.shape
```

[16]:
```
(32000204, 4)
```

[6]:
```
credits_df
```

[6]:

	cast \
0	[{'cast_id': 14, 'character': 'Woody (voice)',…
1	[{'cast_id': 1, 'character': 'Alan Parrish', '…
2	[{'cast_id': 2, 'character': 'Max Goldman', 'c…
3	[{'cast_id': 1, 'character': "Savannah 'Vannah…
4	[{'cast_id': 1, 'character': 'George Banks', '…
…	…
45471	[{'cast_id': 0, 'character': '', 'credit_id': …
45472	[{'cast_id': 1002, 'character': 'Sister Angela…
45473	[{'cast_id': 6, 'character': 'Emily Shaw', 'cr…
45474	[{'cast_id': 2, 'character': '', 'credit_id': …
45475	[]

```
                                                    crew      id
0      [{'credit_id': '52fe4284c3a36847f8024f49', 'de…     862
1      [{'credit_id': '52fe44bfc3a36847f80a7cd1', 'de…    8844
2      [{'credit_id': '52fe466a9251416c75077a89', 'de…   15602
3      [{'credit_id': '52fe44779251416c91011acb', 'de…   31357
4      [{'credit_id': '52fe44959251416c75039ed7', 'de…   11862
…                                                    …     …
45471  [{'credit_id': '5894a97d925141426c00818c', 'de…  439050
45472  [{'credit_id': '52fe4af1c3a36847f81e9b15', 'de…  111109
45473  [{'credit_id': '52fe4776c3a368484e0c8387', 'de…   67758
45474  [{'credit_id': '533bccebc3a36844cf0011a7', 'de…  227506
45475  [{'credit_id': '593e676c92514105b702e68e', 'de…  461257

[45476 rows x 3 columns]
```

[7]:
```
# metadata_df = pd.read_csv(f"{dataset_path}/movies_metadata.csv", nrows=10)#%%
# metadata_df
```

[7]:
```
   adult                              belongs_to_collection    budget  \
0  False  {'id': 10194, 'name': 'Toy Story Collection', …  30000000
1  False                                                NaN  65000000
2  False  {'id': 119050, 'name': 'Grumpy Old Men Collect…         0
3  False                                                NaN  16000000
4  False  {'id': 96871, 'name': 'Father of the Bride Col…         0
5  False                                                NaN  60000000
6  False                                                NaN  58000000
7  False                                                NaN         0
8  False                                                NaN  35000000
9  False  {'id': 645, 'name': 'James Bond Collection', '…  58000000

                                              genres  \
0  [{'id': 16, 'name': 'Animation'}, {'id': 35, '…
1  [{'id': 12, 'name': 'Adventure'}, {'id': 14, '…
2  [{'id': 10749, 'name': 'Romance'}, {'id': 35, …
3  [{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam…
4                    [{'id': 35, 'name': 'Comedy'}]
5  [{'id': 28, 'name': 'Action'}, {'id': 80, 'nam…
6  [{'id': 35, 'name': 'Comedy'}, {'id': 10749, '…
7  [{'id': 28, 'name': 'Action'}, {'id': 12, 'nam…
8  [{'id': 28, 'name': 'Action'}, {'id': 12, 'nam…
9  [{'id': 12, 'name': 'Adventure'}, {'id': 28, '…

                                      homepage     id    imdb_id  \
0  http://toystory.disney.com/toy-story    862  tt0114709
1                                           NaN   8844  tt0113497
2                                           NaN  15602  tt0113228
```

```
3                                                NaN  31357  tt0114885
4                                                NaN  11862  tt0113041
5                                                NaN    949  tt0113277
6                                                NaN  11860  tt0114319
7                                                NaN  45325  tt0112302
8                                                NaN   9091  tt0114576
9  http://www.mgm.com/view/movie/757/Goldeneye/    710  tt0113189

  original_language               original_title  \
0                en                    Toy Story
1                en                      Jumanji
2                en             Grumpier Old Men
3                en            Waiting to Exhale
4                en  Father of the Bride Part II
5                en                         Heat
6                en                      Sabrina
7                en                 Tom and Huck
8                en                 Sudden Death
9                en                    GoldenEye

                                            overview  …  release_date  \
0  Led by Woody, Andy's toys live happily in his …  …    1995-10-30
1  When siblings Judy and Peter discover an encha…  …    1995-12-15
2  A family wedding reignites the ancient feud be…  …    1995-12-22
3  Cheated on, mistreated and stepped on, the wom…  …    1995-12-22
4  Just when George Banks has recovered from his …  …    1995-02-10
5  Obsessive master thief, Neil McCauley leads a …  …    1995-12-15
6  An ugly duckling having undergone a remarkable…  …    1995-12-15
7  A mischievous young boy, Tom Sawyer, witnesses…  …    1995-12-22
8  International action superstar Jean Claude Van…  …    1995-12-22
9  James Bond must unmask the mysterious head of …  …    1995-11-16

     revenue runtime                                  spoken_languages  \
0  373554033    81.0           [{'iso_639_1': 'en', 'name': 'English'}]
1  262797249   104.0  [{'iso_639_1': 'en', 'name': 'English'}, {'iso…
2          0   101.0           [{'iso_639_1': 'en', 'name': 'English'}]
3   81452156   127.0           [{'iso_639_1': 'en', 'name': 'English'}]
4   76578911   106.0           [{'iso_639_1': 'en', 'name': 'English'}]
5  187436818   170.0  [{'iso_639_1': 'en', 'name': 'English'}, {'iso…
6          0   127.0  [{'iso_639_1': 'fr', 'name': 'Français'}, {'is…
7          0    97.0  [{'iso_639_1': 'en', 'name': 'English'}, {'iso…
8   64350171   106.0           [{'iso_639_1': 'en', 'name': 'English'}]
9  352194034   130.0  [{'iso_639_1': 'en', 'name': 'English'}, {'iso…

     status                                          tagline  \
0  Released                                              NaN
1  Released          Roll the dice and unleash the excitement!
```

```
2  Released  Still Yelling. Still Fighting. Still Ready for…
3  Released  Friends are the people who let you be yourself…
4  Released  Just When His World Is Back To Normal… He's …
5  Released                          A Los Angeles Crime Saga
6  Released  You are cordially invited to the most surprisi…
7  Released                            The Original Bad Boys.
8  Released                       Terror goes into overtime.
9  Released              No limits. No fears. No substitutes.

                         title  video vote_average vote_count
0                    Toy Story  False          7.7       5415
1                      Jumanji  False          6.9       2413
2             Grumpier Old Men  False          6.5         92
3            Waiting to Exhale  False          6.1         34
4  Father of the Bride Part II  False          5.7        173
5                         Heat  False          7.7       1886
6                      Sabrina  False          6.2        141
7                Tom and Huck  False          5.4         45
8                Sudden Death  False          5.5        174
9                   GoldenEye  False          6.6       1194

[10 rows x 24 columns]
```

[19]:
```
# Dropping useless bits from movies_meta_df
movies_df = pd.read_csv(f"{dataset_path}/movies_metadata.csv", usecols=['id',␣
 ↪'title', 'release_date', 'genres', 'popularity', 'vote_average',␣
 ↪'vote_count'])
```

```
/var/folders/lr/44zcpqfx2196g_qy5t3kc7m40000gp/T/ipykernel_28406/1038718327.py:2
: DtypeWarning: Columns (10) have mixed types. Specify dtype option on import or
set low_memory=False.
  movies_df = pd.read_csv(f"{dataset_path}/movies_metadata.csv", usecols=['id',
'title', 'release_date', 'genres', 'popularity', 'vote_average', 'vote_count'])
```

[20]:
```
movies_df.shape
```

[20]:
```
(45466, 7)
```

[21]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

[22]:
```
# ratings_small_df = pd.read_csv('/content/drive/MyDrive/Colab_Notebooks/
 ↪Datasets/movie_datasets/7/ratings_small.csv')
```

[23]:
```
# movies_meta_df = pd.read_csv('/content/drive/MyDrive/Colab_Notebooks/Datasets/
 ↪movie_datasets/7/movies_metadata.csv')
# movies_df = pd.read_csv('/content/drive/MyDrive/Colab_Notebooks/Datasets/
 ↪movie_datasets/movies/movies.csv')
```

[24]:
```
# ratings_df = pd.read_csv('/content/drive/MyDrive/Colab_Notebooks/Datasets/
 ↪movie_datasets/movies/ratings.csv')
```

[25]:
```
# credits_df = pd.read_csv('/content/drive/MyDrive/Colab_Notebooks/Datasets/
 ↪movie_datasets/7/credits.csv')
# credits_df.head()
```

[26]:
```
movies_df = movies_df.sort_values(by='release_date', ascending=False)
# movies_meta_df.head(3)
movies_df = movies_df.drop(35587) # A weird film entry is now gone!
movies_df.head(3)
```

[26]:

	genres	id	popularity	release_date	title	vote_average	vote_count
26559	[{'id': 28, 'name': 'Action'}, {'id': 12, 'nam…	76600	6.020055	2020-12-16	Avatar 2	0.0	58.0
38885	[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam…	299782	0.238154	2018-12-31	The Other Side of the Wind	0.0	1.0
30402	[{'id': 53, 'name': 'Thriller'}, {'id': 28, 'n…	38700	2.178546	2018-11-07	Bad Boys for Life	0.0	12.0

3 Preprocessing

[27]:
```
# from math import sqrt
# import numpy as np
# import matplotlib.pyplot as plt

import ast
%matplotlib inline
```

[219]:
```
# prompt: print the dimensions of ratings, credits, movies, movies_metadata

print("ratings_df dimensions:", ratings_df.shape)
print("credits_df dimensions:", credits_df.shape)
print("movies_df dimensions:", movies_df.shape)
# print("movies_meta_df dimensions:", movies_meta_df.shape)
```

```
ratings_df dimensions: (32000204, 4)
credits_df dimensions: (45476, 3)
movies_df dimensions: (45465, 7)
```

[30]:
```
def get_first_3_cast(cast_series):
    """
    Extracts names and IDs of the first 3 cast members from a Pandas Series.

    Args:
```

```
        cast_series (pandas.Series): A Pandas Series containing cast information
                                     (string representation of list of␣
↪dictionaries).

    Returns:
        pandas.DataFrame: DataFrame with a column 'cast_info' containing tuples␣
↪of (name, ID)
                          for the first 3 cast members.
    """

    all_cast_info = []

    for cast_list_str in cast_series:
        cast_list = ast.literal_eval(cast_list_str)  # Convert string to list
        cast_info = []
        for i in range(min(3, len(cast_list))):
            cast_info.append((cast_list[i]['name'], cast_list[i]['id']))  #␣
↪Create (name, ID) tuple
        all_cast_info.append(cast_info)

    return pd.DataFrame({'cast_info': all_cast_info})


# Usage:
cast_info_df = get_first_3_cast(credits_df['cast'])
cast_info_df.head()
```

[30]:

	cast_info
0	[(Tom Hanks, 31), (Tim Allen, 12898), (Don Ric…
1	[(Robin Williams, 2157), (Jonathan Hyde, 8537)…
2	[(Walter Matthau, 6837), (Jack Lemmon, 3151), …
3	[(Whitney Houston, 8851), (Angela Bassett, 978…
4	[(Steve Martin, 67773), (Diane Keaton, 3092), …

[31]:
```
def get_directors_from_crew(film_credits):
    """
    Extracts the names of the director(s) from the 'crew' column of a Pandas␣
↪DataFrame.

    Args:
        film_credits (pandas.DataFrame): A Pandas DataFrame containing 'crew'␣
↪column with crew information.

    Returns:
        pandas.DataFrame: DataFrame with a column 'director_name' containing␣
↪the names of the director(s), and the corresponding director 'id', and␣
↪'credits id' for the film.
```

```
    """
    # Set 'id' column as index
    global director_credit_id
    film_credits = film_credits.set_index('id')

    director_info = []

    for index, row in film_credits.iterrows():  # Iterate using index and row
        crew_list_str = row['crew']
        crew_list = ast.literal_eval(crew_list_str)
        director_name = None
        director_id = None
        film_id = index  # Get the film 'id' (now index)

        for crew_member_l in crew_list:
            if crew_member_l['job'] == 'Director':
                director_name = crew_member_l['name']
                director_credit_id = crew_member_l['credit_id']
                break

        director_info.append((director_name, director_credit_id, film_id))

    return pd.DataFrame({'director_info': director_info})


director_info_df = get_directors_from_crew(credits_df)
director_info_df.head()
```

[31]:

	director_info
0	(John Lasseter, 52fe4284c3a36847f8024f49, 862)
1	(Joe Johnston, 52fe44bfc3a36847f80a7c7d, 8844)
2	(Howard Deutch, 52fe466a9251416c75077a89, 15602)
3	(Forest Whitaker, 52fe44779251416c91011acb, 31…
4	(Charles Shyer, 52fe44959251416c75039eef, 11862)

[32]:
```
def get_first_3_crew(crew_series):
    """
    Extracts names and IDs of the first 3 crew members from a Pandas Series.

    Args:
        crew_series (pandas.Series): A Pandas Series containing crew information
                                     (string representation of list of␣
 ↪dictionaries).

    Returns:
```

```
        pandas.DataFrame: DataFrame with columns 'crew_name' and 'crew_id' for␣
 ↪the first 3 crew members.
    """

    all_crew_info = []

    for crew_list_str in crew_series:
        crew_list_s = ast.literal_eval(crew_list_str)  # Convert string to list
        crew_info = []
        for i in range(min(3, len(crew_list_s))):
            crew_info.append((crew_list_s[i]['name'], crew_list_s[i]['job'],␣
 ↪crew_list_s[i]['id']))  # Create (name, job, ID) tuple
        all_crew_info.append(crew_info)

    return pd.DataFrame({'crew_info': all_crew_info})


crew_info_df = get_first_3_crew(credits_df['crew'])
# crew_info_df.head()
```

[33]:
```
# find all the unique jobs of from crew_info_df
# dataframe example: [(John Lasseter, Director, 7879), (Joss Whedon...
unique_jobs = set()
for crew_list in crew_info_df['crew_info']:
    for crew_member in crew_list:
        unique_jobs.add(crew_member[1])
```

[ ]:
```

```

[256]:
```
top_3_credits_df = pd.concat([cast_info_df, director_info_df,␣
 ↪credits_df['id']], axis=1)
top_3_credits_df.head()
```

[256]:
```
                                           cast_info  \
0  [(Tom Hanks, 31), (Tim Allen, 12898), (Don Ric…
1  [(Robin Williams, 2157), (Jonathan Hyde, 8537)…
2  [(Walter Matthau, 6837), (Jack Lemmon, 3151), …
3  [(Whitney Houston, 8851), (Angela Bassett, 978…
4  [(Steve Martin, 67773), (Diane Keaton, 3092), …

                                      director_info     id
0     (John Lasseter, 52fe4284c3a36847f8024f49, 862)    862
1     (Joe Johnston, 52fe44bfc3a36847f80a7c7d, 8844)   8844
2   (Howard Deutch, 52fe466a9251416c75077a89, 15602)  15602
3  (Forest Whitaker, 52fe44779251416c91011acb, 31…  31357
4   (Charles Shyer, 52fe44959251416c75039eef, 11862)  11862
```

[145]:
```
top_3_credits_df.shape
```

[145]:
```
(45476, 3)
```

[35]:
```
# movies_df.head()
# Make a separate table for genres with the genre name and the corresponding id.
  ↪ Take it from the genres column on movies_df
# Create a separate table for genres with the genre name and the corresponding␣
  ↪id
import json

pre_genre_steal_df = movies_df.copy()

def extract_genres():
    genres_set = set()
    for genres_str in movies_df['genres']:
        genres_list = json.loads(genres_str.replace("'", "\""))
        for genre in genres_list:
            genres_set.add((genre['id'], genre['name']))
    return pd.DataFrame(genres_set, columns=['genre_id', 'genre_name'])

genres_df = extract_genres()
drop_entries = [7759, 7760, 7761, 11602, 11176, 33751, 29812, 2883]
genres_df = genres_df[~genres_df['genre_id'].isin(drop_entries)]
genres_df
```

[35]:

	genre_id	genre_name
0	10751	Family
1	27	Horror
2	18	Drama
3	16	Animation
4	53	Thriller
5	80	Crime
6	37	Western
7	9648	Mystery
10	36	History
14	10770	TV Movie
15	10749	Romance
16	12	Adventure
17	99	Documentary
19	10752	War
20	14	Fantasy
21	10402	Music
23	10769	Foreign
24	878	Science Fiction
25	28	Action
27	35	Comedy

[ ]:
```
ohe_movies_df = movies_df.copy()
```

[ ]:
```
# ohe_movies_df.head(3)
```

[ ]:
```
# Bad genres: (7759,GoHands), (7760,BROSTA TV), (7761,Mardock Scramble␣
↪Production Committee), (11602,Vision View Entertainment), (11176,Carousel␣
↪Productions), (33751,Sentai Filmworks), (29812,Telescene Film Group␣
↪Productions)
```

[39]:
```
drop_entries = [7759, 7760, 7761, 11602, 11176, 33751, 29812, 2883] # Remove␣
↪low quality entries
ohe_movies_df['genres'] = ohe_movies_df['genres'].apply(
    lambda x: json.loads(x.replace("'", "\"")) if isinstance(x, str) else x
).apply(
    lambda x: [gen for gen in x if gen['id'] not in drop_entries] if␣
↪isinstance(x, list) else []
)

# List all the genres (validation check - there's no dodgy genres left in)
all_genres = set()
for genres in ohe_movies_df['genres']:
    for genre in genres:
        all_genres.add(genre['name'])
print(all_genres)
```

```
{'Drama', 'Romance', 'Western', 'History', 'TV Movie', 'Comedy', 'Adventure',
'Music', 'Science Fiction', 'Fantasy', 'Family', 'Mystery', 'War', 'Animation',
'Documentary', 'Thriller', 'Horror', 'Action', 'Foreign', 'Crime'}
```

[40]:
```
# Extract genre names
ohe_movies_df['genre_list'] = ohe_movies_df['genres'].apply(lambda x:␣
↪[gen['name'] for gen in x] if isinstance(x, list) else [])
ohe_movies_df.head()
```

[40]:

	genres	id	popularity	release_date	title	vote_average	vote_count
26559	[{'id': 28, 'name': 'Action'}, {'id': 12, 'nam…	76600	6.020055	2020-12-16	Avatar 2	0.0	58.0
38885	[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'nam…	299782	0.238154	2018-12-31	The Other Side of the Wind	0.0	1.0
30402	[{'id': 53, 'name': 'Thriller'}, {'id': 28, 'n…	38700	2.178546	2018-11-07	Bad Boys for Life	0.0	12.0
38130	[{'id': 18, 'name': 'Drama'}, {'id': 10749, 'n…	332283	3.328261	2018-04-25	Mary Shelley	0.0	1.0
44535	[{'id': 18, 'name': 'Drama'}]	412059	0.155147	2018-04-04	Mobile Homes	0.0	1.0

```
                                          genre_list
26559  [Action, Adventure, Fantasy, Science Fiction]
38885                                 [Comedy, Drama]
30402                       [Thriller, Action, Crime]
38130                                [Drama, Romance]
44535                                         [Drama]
```

[41]:
```
# One-hot encode genres
from sklearn.preprocessing import MultiLabelBinarizer

# Initialize MultiLabelBinarizer
mlb = MultiLabelBinarizer()

genre_ohe = pd.DataFrame(mlb.fit_transform(ohe_movies_df['genre_list']),
                         columns=mlb.classes_,
                         index=ohe_movies_df.index)

# Merge back into sample_genres_movies_df
ohe_movies_df = pd.concat([ohe_movies_df, genre_ohe], axis=1)

genre_list_mlb = mlb.classes_
genre_list = ohe_movies_df['genre_list']

# Drop unnecessary columns
ohe_movies_df = ohe_movies_df.drop(columns=['genres', 'genre_list'])

# print(sample_genres_movies_df.head())
ohe_movies_df
```

[41]:
```
           id popularity release_date  \
26559   76600   6.020055   2020-12-16
38885  299782   0.238154   2018-12-31
30402   38700   2.178546   2018-11-07
38130  332283   3.328261   2018-04-25
44535  412059   0.155147   2018-04-04
…         …          …          …
45148  438910   0.001586          NaN
45203  433711    0.00022          NaN
45338  335251        0.0          NaN
45410  449131   0.008903          NaN
45461  439050   0.072051          NaN

                                          title  vote_average  \
26559                                  Avatar 2           0.0
38885  The Other Side of the Wind           0.0
30402                  Bad Boys for Life           0.0
38130                       Mary Shelley           0.0
```

```
44535                                    Mobile Homes           0.0
…                                                 …           …
45148                                 Engineering Red           6.0
45203  All Superheroes Must Die 2: The Last Superhero           4.0
45338                  The Land Where the Blues Began           0.0
45410                                          Aprel           6.0
45461                                         Subdue           4.0

       vote_count  Action  Adventure  Animation  Comedy  …  History  Horror  \
26559        58.0       1          1          0       0  …        0       0
38885         1.0       0          0          0       1  …        0       0
30402        12.0       1          0          0       0  …        0       0
38130         1.0       0          0          0       0  …        0       0
44535         1.0       0          0          0       0  …        0       0
…             …     …         …         …     …  …      …     …
45148         2.0       0          0          0       0  …        0       0
45203         1.0       0          0          0       0  …        0       0
45338         0.0       0          0          0       0  …        0       0
45410         1.0       0          0          0       0  …        0       0
45461         1.0       0          0          0       0  …        0       0

       Music  Mystery  Romance  Science Fiction  TV Movie  Thriller  War  \
26559      0        0        0                1         0         0    0
38885      0        0        0                0         0         0    0
30402      0        0        0                0         0         1    0
38130      0        0        1                0         0         0    0
44535      0        0        0                0         0         0    0
…      …       …       …              …        …       …  …
45148      0        0        0                0         0         0    0
45203      0        1        0                1         0         0    0
45338      0        0        0                0         0         0    0
45410      0        0        0                0         0         0    0
45461      0        0        0                0         0         0    0

       Western
26559        0
38885        0
30402        0
38130        0
44535        0
…          …
45148        0
45203        0
45338        0
45410        0
45461        0
```

```
[45465 rows x 26 columns]
```

[44]:
```
ohe_movies_df.sort_values(by='id')
```

[44]:
```
           id popularity release_date                                title  \
2429      100    4.60786   1998-03-05  Lock, Stock and Two Smoking Barrels
13609   10000   0.281609   1993-12-25            La estrategia del caracol
4435    10001   2.562888   1988-12-15                        Young Einstein
17451  100010   0.769266   1940-12-27                        Flight Command
36946  100017   2.964103   2006-08-06                               Hounded
…         …          …           …                                     …
25652   99946   0.202315   1926-11-06                          Exit Smiling
3767     9995   1.316179   2000-09-06                           Turn It Up
12549    9997   3.840024   2007-11-15                              Gabriel
25079   99977   0.215778   1979-08-10                           Hot Stuff
13106    9999   1.038858   2006-08-23                       The Free Will

       vote_average  vote_count  Action  Adventure  Animation  Comedy  …  \
2429            7.5      1671.0       0          0          0       1  …
13609           7.2         9.0       0          0          0       1  …
4435            4.5        46.0       0          0          0       1  …
17451           6.0         1.0       0          0          0       0  …
36946           4.8         7.0       0          0          0       0  …
…               …           …         …          …          …       …  …
25652           8.5         2.0       0          0          0       1  …
3767            5.0         5.0       1          0          0       0  …
12549           5.0        77.0       1          0          0       0  …
25079           7.8         6.0       0          0          0       1  …
13106           6.6        11.0       0          0          0       0  …

       History  Horror  Music  Mystery  Romance  Science Fiction  TV Movie  \
2429         0       0      0        0        0                0         0
13609        0       0      0        0        0                0         0
4435         0       0      0        0        0                1         0
17451        0       0      0        0        0                0         0
36946        0       0      0        0        0                0         0
…          …       …      …        …        …                …         …
25652        0       0      0        0        0                0         0
3767         0       0      0        0        0                0         0
12549        0       1      0        0        0                1         0
25079        0       0      0        0        0                0         0
13106        0       0      0        0        0                0         0

       Thriller  War  Western
2429          0    0        0
13609         0    0        0
4435          0    0        0
```

```
17451          0    1         0
36946          0    0         0
…          …   …       …
25652          0    0         0
3767           0    0         0
12549          0    0         0
25079          0    0         0
13106          0    0         0

[45465 rows x 26 columns]
```

4 Creating a user profile

[245]:
```
user_ratings = """
710,golden eye, 4
1770,Michael Collins, 5
76600,Avatar 2, 3
141052,Justice League, 2.5
284053,Thor: Ragnarok, 4
341013,Atomic Blonde, 3.8
374720,Dunkirk, 4.7
339403,Baby Driver, 4.7
324852,Despicable Me 3, 4
419192,McLaren, 3.5
283995,Guardians of the Galaxy Vol. 2, 3.8
324849,The Lego Batman Movie, 4.5
324552,John Wick: Chapter 2, 3.5
180863,T2 Trainspotting, 3.6
318846,The Big Short, 4.4
406,La Haine, 4.65
424,Schindler's List, 4.8
627,Trainspotting, 4.9
107,Snatch, 4.9
1429,25th Hour, 3.3
49517,Tinker Tailor Soldier Spy, 4.1
6538,Charlie Wilson's War, 3.9
10315,Fantastic Mr. Fox, 4.7
27205,Inception, 3.6
106646,The Wolf of Wall Street, 4.5
120467,The Grand Budapest Hotel, 4.6
157336,Interstellar, 3.9
261023,Black Mass, 3.6
278,The Shawshank Redemption, 3
4232,Scream, 2.4
1893,Star Wars: Episode I - The Phantom Menace, 3.8
550,Fight Club, 4.2
```

```
98,Gladiator, 4
508,Love Actually, 2.1
228967,The Interview, 2.7
1895,Star Wars: Episode III - Revenge of the Sith, 3.5
18785,The Hangover, 1.9
67913,The Guard, 3.7
40807,50/50, 2.5
70160,The Hunger Games, 2.4
77930,Magic Mike, 1
72190,World War Z, 2.5
187017,22 Jump Street, 1.9
198663,The Maze Runner, 2
207703,Kingsman: The Secret Service, 2.1
99861,Avengers: Age of Ultron, 2
167073,Brooklyn, 2.8
254470,Pitch Perfect 2, 1.9
271718,Trainwreck, 1
314365,Spotlight, 4.9
259693,The Conjuring 2, 2.3
308266,War Dogs, 2.1
324786,Hacksaw Ridge, 3.1
330459,Rogue One: A Star Wars Story, 2.9
339846,Baywatch, 2
"""

len(user_ratings)

# 100,"Lock, Stock and Two Smoking Barrels", 4.3
```

[245]: 1465

[221]:
```
from io import StringIO

# Convert into a DataFrame
user_ratings_df = pd.read_csv(StringIO(user_ratings), header=None,␣
 ↪names=["movieId", "movie_name", "rating"])

# Drop the movie_name column as it's not needed for appending to the ratings␣
 ↪DataFrame
user_ratings_df = user_ratings_df.drop(columns=["movie_name"])

user_ratings_df['movieId'] = pd.to_numeric(user_ratings_df['movieId'],␣
 ↪errors='coerce')
user_ratings_df = user_ratings_df.dropna(subset=['movieId'])
user_ratings_df['movieId'] = user_ratings_df['movieId'].astype(int)

# Testing user id = 999,999
```

```
user_ratings_df["userId"] = 999999

# Reorder columns to match the existing ratings DataFrame
user_ratings_df = user_ratings_df[["userId", "movieId", "rating"]]
```

```
   userId  movieId  rating
0  999999      710     4.0
1  999999     1770     5.0
2  999999    76600     3.0
```

[222]:
```
ratings_df = pd.concat([ratings_df, user_ratings_df], ignore_index=True)
# ratings_df = ratings_df.drop('timestamp', axis=1)
ratings_df.head()
```

[222]:
```
   userId  movieId  rating    timestamp
0       1       17     4.0  944249077.0
1       1       25     1.0  944250228.0
2       1       29     2.0  943230976.0
3       1       30     5.0  944249077.0
4       1       32     5.0  943228858.0
```

[ ]:
```
genre_normalisation = 0.12

def create_user_profile(user_id, ratings_df, top_3_credits_df):
  """Creates a user profile based on ratings for movies with shared cast/
 ↪directors."""
  directors = []
  user_ratings = ratings_df[ratings_df['userId'] == user_id]
  # print(user_ratings)
  profile = {}

  for _, rating_row in user_ratings.iterrows():
    movie_id = rating_row['movieId']
    rating = rating_row['rating']

    #find the index of the movie_id in top_3_credits_df
    try:
        movie_index = top_3_credits_df[top_3_credits_df['id'] == movie_id].
 ↪index[0]
    except IndexError:
        print(f"Movie ID {movie_id} not found in top_3_credits_df")
        continue # if the movie_id isn't in top_3_credits_df, skip this movie


    # Add cast and director IDs to user profile with weighted ratings
    cast_ids = top_3_credits_df['cast_info'].iloc[movie_index]
    for cast_member in cast_ids:
```

```
        profile[cast_member[1]] = profile.get(cast_member[1], 0) + rating

    director_info = top_3_credits_df['director_info'].iloc[movie_index]
    if director_info is not None:
        profile[director_info[0]] = profile.get(director_info[0], 0) + rating
        directors.append(director_info[0])

    # Add genre IDs to user profile with weighted ratings
    genre_score = genre_normalisation * rating
    for film_genre in mlb.classes_:
        if ohe_movies_df[film_genre].iloc[movie_index] == 1:
            profile[film_genre] = profile.get(film_genre, 0) + genre_score


  return profile, directors
```

[ ]:
```
test_user_id = 999999  # Test User ID (Cathal's great taste in movies)
user_profile, directors_list = create_user_profile(test_user_id, ratings_df,␣
 ↪top_3_credits_df)
user_profile
```

4.0.1 User User Collaborative Filtering

[261]:
```
ratings_cpy = ratings_df.copy()
```

[262]:
```
# # Sort ratings_df by timestamp (most recent) and then translate it into a␣
 ↪data format
# ratings_cpy = ratings_cpy.sort_values(by='timestamp', ascending=False)
# ratings_cpy['date'] = pd.to_datetime(ratings_cpy['timestamp'], unit='s')
# ratings_cpy.head()
```

[264]:
```
from lenskit.algorithms import Recommender
from lenskit.algorithms.user_knn import UserUser
```

[265]:
```
ratings_cpy = ratings_cpy.drop('timestamp', axis=1)
```

[228]:
```
num_recs = 100  #<---- This is the number of recommendations to generate. You␣
 ↪can change this if you want to see more recommendations

ratings_cpy.rename(columns={'userId': 'user'}, inplace=True)
ratings_cpy.rename(columns={'movieId': 'item'}, inplace=True)

user_user = UserUser(20, min_nbrs=2) #These two numbers set the minimum (3) and␣
 ↪maximum (15) number of neighbours to consider. These are considered␣
 ↪"reasonable defaults", but you can experiment with others too
algo = Recommender.adapt(user_user)
algo.fit(ratings_cpy)
```

```
print("User-User algorithm set up!")
```

```
Set up a User-User algorithm!
```

[266]:
```
def get_user_user_recs(user_id, num_ids = 400):
    return algo.recommend(user_id, num_ids)
```

[297]:
```
cathal_recs = algo.recommend(999999, num_recs)  #Here, -1 tells it that it's␣
↪not an existing user in the set, that we're giving new ratings, while 10 is␣
↪how many recommendations it should generate

# Print recommended movies
# cathal_recs
```

[297]:
```
Empty DataFrame
Columns: [title, release_date, vote_average, vote_count]
Index: []
```

[299]:
```
ohe_movies_df[ohe_movies_df['id'].isin(cathal_recs.item)][['title',␣
↪'release_date', 'vote_average', 'vote_count']]
```

[299]:

	title	release_date	vote_average	vote_count
34833	La mossa del pinguino	2014-03-01	5.3	25.0
26127	Grave Halloween	2013-10-19	4.4	28.0
41347	Love 911	2012-12-19	6.8	14.0
40422	Rat Fever	2012-06-22	0.0	0.0
33070	The Saint of Gamblers	1995-06-28	3.5	2.0
38618	The Gypsy and the Gentleman	1958-01-15	0.0	0.0
32146	K.O. Miguel	1957-01-01	0.0	0.0
19433	We Live Again	1934-11-01	0.0	0.0

[300]:
```
save_ohe_movies_df = ohe_movies_df.copy()
print("Dimensions of ohe_movies_df before merge:", save_ohe_movies_df.shape)
print("Dimensions of top_3_credits_df before merge:", top_3_credits_df.shape)

# Convert the 'id' columns to numeric, forcing errors to NaN
ohe_movies_df.loc[:, 'id'] = pd.to_numeric(ohe_movies_df['id'], errors='coerce')
top_3_credits_df['id'] = pd.to_numeric(top_3_credits_df['id'], errors='coerce')

# Drop rows with NaN values in the 'id' columns
ohe_movies_df = ohe_movies_df.dropna(subset=['id'])
top_3_credits_df = top_3_credits_df.dropna(subset=['id'])

# Convert the 'id' columns to integers
ohe_movies_df.loc[:, 'id'] = ohe_movies_df['id'].astype(int)
top_3_credits_df['id'] = top_3_credits_df['id'].astype(int)
```

```
# Find common IDs
# common_ids = set(ohe_movies_df['id']) & set(top_3_credits_df['id'])
# print("Number of matching movie IDs:", len(common_ids))
# print("Number of matching movie IDs:", len(set(ohe_movies_df['id']) &␣
 ↪set(top_3_credits_df['id'])))

# Merge the two DataFrames
# ohe_movies_df = pd.merge(ohe_movies_df, top_3_credits_df, on='id',␣
 ↪how='inner')

print("Dimensions of ohe_movies_df after merge:", ohe_movies_df.shape)
```

```
Dimensions of ohe_movies_df before merge: (45463, 26)
Dimensions of top_3_credits_df before merge: (45476, 3)
Dimensions of ohe_movies_df after merge: (45463, 26)
```

5 Generate recommendations

[271]:
```
def score_breakdown(films_df, recommended_movies):
    return films_df[films_df['id'].isin([movie_id for movie_id, _, _, _, _, _␣
 ↪in recommended_movies])][['title', 'release_date', 'vote_average',␣
 ↪'vote_count']].assign(
    score=[score for _, score, _, _, _, _ in recommended_movies],
    cast_score=[cast_score for _, _, cast_score, _, _, _ in recommended_movies],
    director_score=[director_score for _, _, _, director_score, _, _ in␣
 ↪recommended_movies],
    genre_score=[genre_score for _, _, _, _, genre_score, _ in␣
 ↪recommended_movies],
    user_user_score=[user_user_score for _, _, _, _, _, user_user_score in␣
 ↪recommended_movies]
    )
```

5.0.1 Content-Based Filtering

[278]:
```
# I want to see how many movies actually have a director that matches with the␣
 ↪user profile directors
matching_directors_count = top_3_credits_df[top_3_credits_df['director_info'].
 ↪apply(lambda x: x[0] if x is not None else None).isin(directors_list)].
 ↪shape[0]
print(f"Number of movies with matching directors: {matching_directors_count}")
```

```
Number of movies with matching directors: 478
```

[286]:
```
cast_ft_weight = 0.3
director_ft_weight = 0.3
genre_ft_weight = 0.15
```

```
user_user_weight = 1

def recommend_movies(user_id, user_profile, films, top_3_credits_df, directors,␣
↪ratings_df, top_n=20):
    """Recommends movies based on user profile and movie cast/director."""

    user_rated_movies = set(ratings_df[ratings_df['userId'] ==␣
↪user_id]['movieId'])
    unrated_movies = films[~films['id'].isin(user_rated_movies)]

    print("Unrated movies dimensions:", unrated_movies.shape)
    recommendations = []

    user_user_scores = get_user_user_recs(user_id, 900)
    print("User-User scores retrieved!")

    for _, movie_row in unrated_movies.iterrows():
        movie_id = movie_row['id']
        score, cast_score, director_score, genre_score, user_user_score = 0, 0,␣
↪0, 0, 0

        # Append on user-user collaborative filtering score
        if movie_id in user_user_scores.item.values:
            user_user_score = user_user_scores[user_user_scores['item'] ==␣
↪movie_id].score.values[0]

        score += user_user_score * user_user_weight

        #find the index of the movie_id in top_3_credits_df
        try:
            movie_index = top_3_credits_df[top_3_credits_df['id'] == movie_id].
↪index[0]
        except IndexError:
            print(f"Movie ID {movie_id} not found in top_3_credits_df")
            continue

        # Calculate weighted score based on user profile and movie cast/
↪director, genres,
        cast_ids = top_3_credits_df['cast_info'].iloc[movie_index]
        for cast_member in cast_ids:
            cast_score += user_profile.get(cast_member[1], 0) * cast_ft_weight
        score += cast_score

        director_info = directors.iloc[movie_index]
        director_score += user_profile.get(director_info[0], 0) *␣
↪director_ft_weight
```

```
        score += director_score

        # Calculate weighted score based on one hot-encoded genres
        # TODO - need to make this a normalised score - e.g. if you have 7␣
↪genres that all, you're greatly dominating other films with 1 genre that␣
↪matches
        genre_score = sum([user_profile.get(film_genre, 0) for film_genre in␣
↪mlb.classes_ if movie_row[film_genre] == 1])
        score += genre_score * genre_ft_weight

        recommendations.append((movie_id, score, cast_score, director_score,␣
↪genre_score, user_user_score))

    recommendations.sort(key=lambda x: x[1], reverse=True)  # Sort by score
    top_recommendations = recommendations[:top_n]

    top_recommendations = [(movie_id, score, cast_score, director_score,␣
↪genre_score, user_user_score)
                           for movie_id, score, cast_score, director_score,␣
↪genre_score, user_user_score in top_recommendations]

    # # recommendations.sort(key=lambda x: x[1])  # Sort in ascending order
    # lowest_recommendations = recommendations[-top_n:]
    # least_movie_recs = [(movie_id, score, cast_score, director_score,␣
↪genre_score, user_user_score) for movie_id, score, cast_score,␣
↪director_score, genre_score, user_user_score in lowest_recommendations]

    # return movie_recs, least_movie_recs
    return top_recommendations
```

[287]:
```
recommended_movies, least_rated_movie_ids = recommend_movies(test_user_id,␣
↪user_profile, ohe_movies_df, top_3_credits_df,␣
↪top_3_credits_df['director_info'], ratings_df, top_n=50)
```

```
Unrated movies dimensions: (45408, 26)
User-User scores retrieved!
Movie ID 401840 not found in top_3_credits_df
```

[292]:
```
score_breakdown(ohe_movies_df, recommended_movies)
```

[292]:

	title	release_date
36242	Miles Ahead	2016-03-20
35696	Jane Got a Gun	2016-01-01
28624	Run All Night	2015-03-11
27477	Mortdecai	2015-01-21
25857	Son of a Gun	2014-10-16
24640	Foxcatcher	2014-05-19

```
20829                                This Is the End   2013-06-12
20721                                       Trance   2013-03-27
41347                                     Love 911   2012-12-19
18252                           The Dark Knight Rises   2012-07-16
19016                                Moonrise Kingdom   2012-05-16
18779                              Wrath of the Titans   2012-03-27
17758                                       Moneyball   2011-09-22
24472           National Theatre Live: Frankenstein   2011-03-17
18559                                   Perfect Sense   2011-01-24
17237                                       Beginners   2010-09-11
15344                                     The A-Team   2010-06-10
15006                              Clash of the Titans   2010-04-01
14825                                   Shutter Island   2010-02-18
14845                                The Ghost Writer   2010-02-12
14141                     The Men Who Stare at Goats   2009-10-17
15183                                   The Good Heart   2009-09-11
13670                                Angels & Demons   2009-05-13
14792                      I Love You Phillip Morris   2009-01-18
13219           The Curious Case of Benjamin Button   2008-11-24
13000                                   Body of Lies   2008-10-10
12481                                The Dark Knight   2008-07-16
24007                                       Beautiful   2008-02-14
13690                                     Incendiary   2008-01-20
22526                                Hotel Chevalier   2007-09-03
11354                                   The Prestige   2006-10-19
10474                                           Stay   2005-09-24
10334                                Just Like Heaven   2005-09-16
10567                              Breakfast on Pluto   2005-09-03
10122                                   Batman Begins   2005-06-10
7914          Anchorman: The Legend of Ron Burgundy   2004-07-09
7275                                      Young Adam   2003-09-26
6223                                   Down with Love   2003-05-08
5244   Star Wars: Episode II - Attack of the Clones   2002-05-15
3167                                        The Beach   2000-02-11
3008                             The End of the Affair   1999-12-03
3120                               Eye of the Beholder   1999-09-04
2515                                   The Love Letter   1999-05-21
2224                                   Velvet Goldmine   1998-10-23
1580                             A Life Less Ordinary   1997-10-24
1307                                       Nightwatch   1997-01-31
1472                                     Brassed Off   1996-11-01
111                                  Before and After   1996-02-23
33070                            The Saint of Gamblers   1995-06-28
315                                     Shallow Grave   1994-12-22

       vote_average  vote_count      score  cast_score  director_score  \
36242           6.7        74.0  10.884000       5.400            3.66
```

35696	5.4	293.0	8.626500	2.550	3.66
28624	6.3	1169.0	7.396500	1.320	3.66
27477	5.4	1078.0	7.093500	1.320	3.66
25857	6.1	284.0	7.594500	3.600	2.79
24640	6.5	965.0	7.439250	2.250	2.79
20829	6.2	2394.0	10.608750	4.740	2.55
20721	6.5	975.0	8.149500	4.740	2.55
41347	6.8	14.0	7.660500	3.630	2.55
18252	7.6	9263.0	7.465500	2.535	2.55
19016	7.6	1701.0	7.438500	2.430	2.55
18779	5.5	1459.0	11.128500	7.980	2.19
17758	7.0	1409.0	7.544250	4.080	1.50
24472	8.5	15.0	6.936000	4.320	1.50
18559	6.9	298.0	9.403500	5.940	1.35
17237	6.8	353.0	6.921220	0.000	1.32
15344	6.2	1737.0	7.143000	3.210	1.26
15006	5.6	2280.0	7.609500	4.260	1.20
14825	7.8	6559.0	7.019250	4.920	0.81
14845	6.7	681.0	8.923500	6.060	0.75
14141	5.9	755.0	9.673500	8.010	0.00
15183	6.0	21.0	9.610500	8.730	0.00
13670	6.5	2183.0	9.045000	8.730	0.00
14792	6.4	514.0	9.043500	7.290	0.00
13219	7.3	3398.0	7.869390	2.070	0.00
13000	6.5	919.0	7.836000	6.720	0.00
12481	8.3	12269.0	7.659000	4.740	0.00
24007	6.6	8.0	7.482947	0.000	0.00
13690	5.3	55.0	7.264500	6.060	0.00
22526	7.1	204.0	7.198500	5.490	0.00
11354	8.0	4510.0	7.168500	5.460	0.00
10474	6.5	346.0	7.156500	4.740	0.00
10334	6.5	595.0	7.150500	4.740	0.00
10567	7.0	77.0	7.139250	4.740	0.00
10122	7.5	7511.0	7.139250	4.740	0.00
7914	6.7	1523.0	7.139250	4.740	0.00
7275	5.9	46.0	7.115250	6.150	0.00
6223	6.1	202.0	7.113750	4.650	0.00
5244	6.4	4074.0	7.107149	0.000	0.00
3167	6.3	1271.0	7.056750	4.740	0.00
3008	6.6	56.0	7.054500	5.850	0.00
3120	5.3	42.0	7.042806	0.000	0.00
2515	5.9	17.0	6.983250	5.820	0.00
2224	6.9	119.0	6.937500	5.940	0.00
1580	6.2	130.0	6.907500	5.910	0.00
1307	6.2	104.0	6.901500	4.080	0.00
1472	6.9	53.0	6.887713	0.000	0.00
111	5.8	37.0	6.883500	4.740	0.00

33070	3.5	2.0	6.855755	0.000	0.00
315	7.0	247.0	6.853500	4.740	0.00

	genre_score	user_user_score
36242	12.160	0.000000
35696	16.110	0.000000
28624	16.110	0.000000
27477	14.090	0.000000
25857	8.030	0.000000
24640	15.995	0.000000
20829	22.125	0.000000
20721	5.730	0.000000
41347	9.870	0.000000
18252	15.870	0.000000
19016	16.390	0.000000
18779	6.390	0.000000
17758	13.095	0.000000
24472	7.440	0.000000
18559	14.090	0.000000
17237	5.655	4.752970
15344	17.820	0.000000
15006	14.330	0.000000
14825	8.595	0.000000
14845	14.090	0.000000
14141	11.090	0.000000
15183	5.870	0.000000
13670	2.100	0.000000
14792	11.690	0.000000
13219	9.385	4.391640
13000	7.440	0.000000
12481	19.460	0.000000
24007	15.995	5.083697
13690	8.030	0.000000
22526	11.390	0.000000
11354	11.390	0.000000
10474	16.110	0.000000
10334	16.070	0.000000
10567	15.995	0.000000
10122	15.995	0.000000
7914	15.995	0.000000
7275	6.435	0.000000
6223	16.425	0.000000
5244	14.830	4.882649
3167	15.445	0.000000
3008	8.030	0.000000
3120	9.335	5.642556
2515	7.755	0.000000

2224	6.650	0.000000
1580	6.650	0.000000
1307	18.810	0.000000
1472	15.995	4.488463
111	14.290	0.000000
33070	15.995	4.456505
315	14.090	0.000000

Next, let's look at the ratings dataframe.